

课程笔记

CS231n Lecture 8: Attention and Transformers

Codex Harness

2026-04-15



视频作者/频道: Stanford Online / CS231n

发布日期: 2025-04-24

视频时长: 01:07:16

视频链接: https://www.youtube.com/watch?v=RQowi0F_FvQ

目录

1 从序列到序列到注意力: 为什么瓶颈会出现	2
1.1 本章小结	2
2 Cross-attention: 用可学习检索替代固定长度压缩	2
2.1 本章小结	4
3 Self-attention 的矩阵视角	4
3.1 本章小结	4
4 位置编码、mask 与多头机制	4
4.1 本章小结	5
5 Transformer block、训练目标与推理方式	5
5.1 本章小结	6
6 Vision Transformer 与近期结构 tweak	6
6.1 本章小结	7
7 总结与延伸	7

1 从序列到序列到注意力: 为什么瓶颈会出现

上一讲的 RNN 告诉我们, 序列建模可以通过 hidden state 把历史信息递推下去。但这讲一开始就把一个关键问题摆到了台面上: 如果输入和输出都很长, 只靠一个固定长度的上下文向量, decoder 往往会被迫在“记住一切”和“立刻生成下一步”之间做不可能的折中。

在 sequence-to-sequence (seq2seq) 模型中, 经典 encoder-decoder 结构会先把整条输入序列编码成最后一个状态, 再把这个状态交给 decoder 逐步生成输出。这个方案的短板很明显:

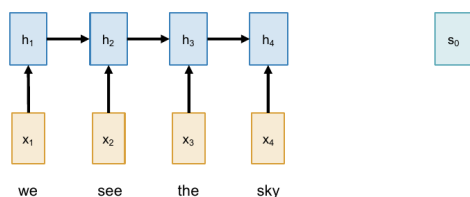
- 输入越长, 最后一个状态承载的信息越拥挤。
- decoder 在第 t 步生成时, 无法有选择地回看输入中的局部片段。
- 某些局部对齐关系, 例如翻译中的词对齐或视觉描述中的局部区域对齐, 很难通过单个向量精确表达。

Sequence to Sequence with RNNs and Attention

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

Encoder: $h_t = f_W(x_t, h_{t-1})$ From final hidden state:
Initial decoder state s_0



Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Stanford CS231n 10th Anniversary

Lecture 8 - 13

April 24, 2025

图 1: Attention 最初是为了解决 encoder-decoder 的固定瓶颈: decoder 需要按步访问 encoder 中不同位置的信息。

因此, 课程把 attention 定义为一种新原语: 与其把所有信息预先压缩成一个向量, 不如让当前步骤主动去“检索”它需要的上下文。

1.1 本章小结

attention 的出发点不是替换一层算子, 而是推翻“所有历史必须压进单一上下文向量”的假设。它让模型从固定压缩转向按需读取。

2 Cross-attention: 用可学习检索替代固定长度压缩

如果把 decoder 当前状态记作 query, 把 encoder 的隐藏状态记作 keys 和 values, 那么 attention 的基本步骤可以写成三步:

$$e_{i,j} = \frac{q_i^\top k_j}{\sqrt{d_k}}$$

$$a_{i,j} = \text{softmax}_j(e_{i,j})$$

$$y_i = \sum_j a_{i,j} v_j$$

其中：

- q_i : 第 i 个输出位置对应的 query。
- k_j : 第 j 个输入位置对应的 key。
- v_j : 第 j 个输入位置对应的 value。
- $e_{i,j}$: query 与 key 的匹配分数。
- $a_{i,j}$: 归一化后的注意力权重。
- y_i : 当前输出位置聚合得到的上下文向量。

这个形式揭示了 attention 的本质：当前步骤不再被动接受一个固定上下文，而是显式地在所有输入位置上分配概率质量。

当有多个 query 一起计算时，上面的公式可以矩阵化：

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \quad Y = AV$$

这里 A 的每一行或每一列（取决于实现约定）都表示一个 query 对所有 key 的分布。课程在 slides 里特意把这一层层展开，是为了让读者看到：attention 并不神秘，它就是“先算相似度，再做 softmax，再用它对 value 做加权”。

Self-Attention Layer

Inputs:

Input vectors: X [$N \times D_{in}$]

Key matrix: W_K [$D_{in} \times D_{out}$]

Value matrix: W_V [$D_{in} \times D_{out}$]

Query matrix: W_Q [$D_{in} \times D_{out}$]

Each input produces one output, which is a mix of information from all inputs

Computation:

Queries: $Q = XW_Q$ [$N \times D_{out}$]

Keys: $K = XW_K$ [$N \times D_{out}$]

Values: $V = XW_V$ [$N \times D_{out}$]

Similarities: $E = QK^\top / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ [$N \times N$]

Output vector: $Y = AV$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

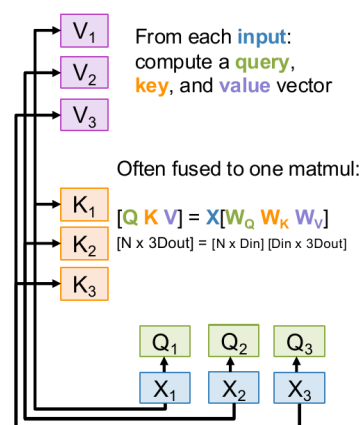


图 2: Attention matrix 可以理解为 query 对 key 的检索权重表，而输出是这些权重对 values 的线性组合。

为什么要除以 $\sqrt{d_k}$

课程强调，如果点积的尺度太大，softmax 容易进入过饱和区间，梯度就会变差。缩放因子 $\sqrt{d_k}$ 的角色，就是把分数维持在更稳定的数值范围内。

2.1 本章小结

cross-attention 允许 decoder 针对不同输出位置读取不同输入位置，从而把“上下文”从固定向量改造成动态检索结果。

3 Self-attention 的矩阵视角

当 query、key、value 都来自同一条输入序列时，attention 就变成了 **self-attention**。设输入为 $X \in \mathbb{R}^{N \times D}$ ，那么：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$
$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \quad Y = AV$$

这套写法非常关键，因为它直接把序列内部的所有 pairwise interaction 统一成矩阵乘法。课程里反复展示 attention weights，目的就是让学生从“递归式记忆”切换到“全局关系图”的视角：每个 token 都可以直接和其它 token 建立联系。

与 RNN 的根本区别

RNN 依赖状态链逐步传递信息；self-attention 则让任意两个位置在单层里直接交互。前者是路径长度随距离增长，后者是路径长度近似常数，但代价是 $N \times N$ 的关系矩阵。

这也解释了为什么课程同时强调 self-attention 的能力与代价：

- 好处：长程依赖不需要穿过很长的状态链。
- 代价：时间和显存都随序列长度平方增长。

3.1 本章小结

self-attention 把“序列处理”变成了“关系矩阵计算”。这让模型更容易捕捉远程依赖，但也引入了二次复杂度的现实约束。

4 位置编码、mask 与多头机制

单纯的 self-attention 有一个严重问题：如果不加入位置信息，模型看到的只是一组无序 token，交换 token 顺序并不会改变 pairwise similarity 的集合。因此，课程把 **位置编码 (positional encoding)** 作为 Transformer 的必要补丁，而不是锦上添花。

另一项关键机制是 **masked self-attention**。在自回归生成里，第 t 个位置只能访问自己之前的 token，不能偷看未来。因此 mask 的角色，是在 softmax 之前把不该看的位置强行压成无穷小概率。

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: X $[N \times D]$
 Key matrix: W_K $[D \times HD_H]$
 Value matrix: W_V $[D \times HD_H]$
 Query matrix: W_Q $[D \times HD_H]$
 Output matrix: W_O $[HD_H \times D]$

Computation:

Queries: $Q = XW_Q$ $[H \times N \times D_H]$
 Keys: $K = XW_K$ $[H \times N \times D_H]$
 Values: $V = XW_V$ $[H \times N \times D_H]$
 Similarities: $E = QK^T / \sqrt{D_Q}$ $[H \times N \times N]$
 Attention weights: $A = \text{softmax}(E, \text{dim}=2)$ $[H \times N \times N]$
 Head outputs: $Y = AV$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$
 Outputs: $O = YW_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$
 Split and reshape to get Q, K, V each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H]$ $[H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V-Weighting

$[H \times N \times N]$ $[H \times N \times D_H] \Rightarrow [H \times N \times D_H]$
 Reshape to $[N \times HD_H]$

4. Output Projection

$[N \times HD_H]$ $[HD_H \times D] \Rightarrow [N \times D]$

Q: How much memory does this take as the number of vectors N increases?

Stanford CS231n 10th Anniversary

Lecture 8 - 75

April 24, 2025

图 3: 多头注意力和 mask 一起定义了 Transformer 的上下文可见性与表征分工。

多头注意力 (multi-head self-attention) 则进一步回答了另一个问题: 如果一套 $Q/K/V$ 投影只能在一个子空间里建模关系, 是否会太单一? 课程给出的答案是并行运行 H 份注意力, 再把结果拼接起来:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O$$

每个 head 都有自己的一组投影矩阵, 因此它们可以分别捕捉不同粒度、不同语义的依赖。工程上看, 多头机制把大一次投影拆成多个小子空间; 建模上看, 它避免了“所有关系都挤在一张 attention map 上”。

4.1 本章小结

位置编码解决顺序感缺失, mask 解决自回归信息泄露, 多头机制解决单一注意力空间过于拥挤的问题。三者合在一起, self-attention 才真正可用。

5 Transformer block、训练目标与推理方式

Transformer 真正强大的地方, 不只是 self-attention 这一个层, 而是把注意力、前馈网络、残差连接和归一化组织成了可重复堆叠的 block。

```
1 def self_attention_block(X):
2     Q = X @ W_Q
3     K = X @ W_K
4     V = X @ W_V
5     A = softmax((Q @ K.T) / sqrt(d_k), dim=-1)
6     Y = A @ V
7     X = layer_norm(X + Y)
8     Z = mlp(X)
9     return layer_norm(X + Z)
```

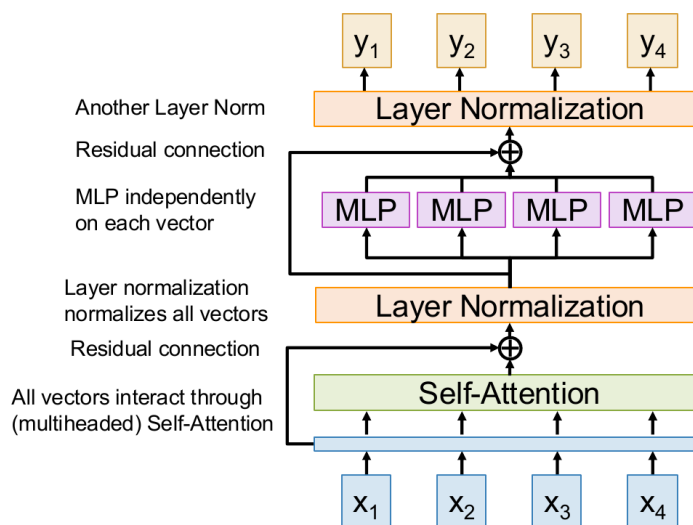
Listing 1: 简化版 Transformer block / self-attention 伪代码

课程里专门用 “Self-Attention is Four Matrix Multiplies!” 来强调：从实现角度看，Transformer 的主体可以高度规约成矩阵乘法 and 简单逐元素操作，这正是它后来能在 GPU/TPU 上被大规模并行化的原因之一。

The Transformer

Transformer Block

Input: Set of vectors x



Vaswani et al, "Attention is all you need," NeurIPS 2017

Stanford CS231n 10th Anniversary

Lecture 8 - 90

April 24, 2025

图 4: Transformer block 把注意力、残差、归一化与 MLP 拼成标准模块，使模型可以稳定地向深层堆叠。

训练目标方面，课程延续的是 next-token prediction：给定前缀，预测下一个 token 的分布。这一目标可以落在 encoder-decoder、decoder-only 等多种结构上；区别主要在于：

- encoder-decoder 更适合 “输入序列 $\rightarrow$ 输出序列” 的映射。
- decoder-only 更适合统一的自回归建模。
- masked self-attention 保证训练时目标与推理时可见信息一致。

5.1 本章小结

Transformer 的成功来自 “新原语 + 可扩展 block 结构 + 适合并行硬件” 的组合，而不是单一公式。attention 提供表达力，block 提供工程可扩展性。

6 Vision Transformer 与近期结构 tweak

本讲最后把注意力正式带回视觉任务。Vision Transformer (ViT) 的关键动作是把图像切成 patch，并把每个 patch 当成一个 token 输入 Transformer。这样一来，NLP 里的 token sequence 被替换成了视觉 patch sequence，但 block 的核心形式几乎不变。

Vision Transformers (ViT)

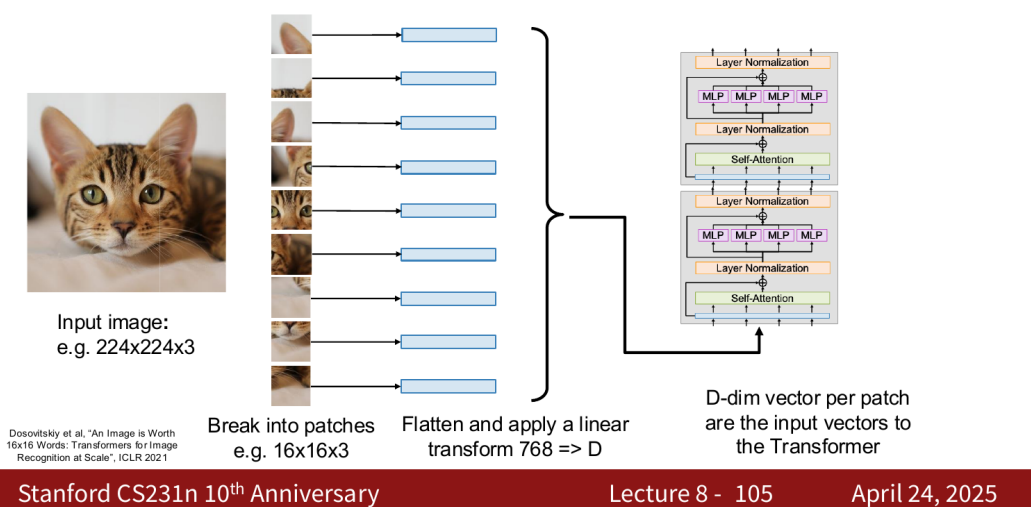


图 5: ViT 把图像 patchify 成 token，再用 Transformer 建模全局视觉关系。

与此同时，课程还快速回顾了若干“Transformer 体系结构没有根本变化，但细部实现持续演进”的 tweak：

- **Pre-Norm**：把归一化前置，提高深层训练稳定性。
- **RMSNorm**：以更轻量化的方式做尺度控制。
- **SwiGLU**：在 MLP 中使用更强的门控非线性。
- **MoE**：让部分前馈子模块稀疏激活，提高参数规模与算力利用的折中。

这些 tweak 说明，Transformer 的中心骨架很稳定，但在数值稳定性、计算效率与容量扩展上，仍然有很多工程优化空间。

6.1 本章小结

ViT 说明 attention 并不局限于文本，只要能把输入离散成 token 序列，就可以把 Transformer 迁移过去。而近期 tweak 则说明，成熟架构的演进更多体现在稳定性和效率。

7 总结与延伸

这讲完成了一个非常重要的转折：从“用状态链传递历史”的 RNN 视角，切换到“用可学习检索和关系矩阵处理上下文”的 attention / Transformer 视角。核心收获可以概括为三点：

- attention 解决了 encoder-decoder 的固定瓶颈，让模型能按需回看输入。
- self-attention 把序列内部依赖统一成矩阵运算，为并行训练打下了基础。
- Transformer block 把注意力、MLP、残差和归一化组织成了可堆叠、可扩展、可迁移到视觉的标准模块。

对计算机视觉而言，ViT 的意义不只是“把 Transformer 搬到图像上”，而是提示我们：一旦 tokenization 设计得当，很多视觉任务也可以享受 attention 带来的全局建模能力。下一讲进入 detection、segmentation 和 model interpretation 时，这种“全局关系建模”的思想会进一步和空间预测任务结合起来。